

PENJELASAN DETAIL SETIAP BARIS KODE

Program Bilangan Prima C++ (Analisis Line-by-Line)

Baris 1

```
#include <iostream>
```

Fungsi:

Menyertakan library (pustaka) iostream ke dalam program

Penjelasan Detail:

- **#include** adalah preprocessor directive yang memberitahu compiler untuk menyalin isi file header
- **iostream** = Input Output Stream
- Library ini berisi fungsi untuk input (cin) dan output (cout)
- Tanpa ini, cout dan cin tidak akan berfungsi

Analogi:

Seperti meminjam buku dari perpustakaan sebelum bisa membacanya. iostream adalah "buku" yang berisi cara berkomunikasi dengan pengguna.

Baris 2

```
using namespace std;
```

Fungsi:

Memberitahu compiler untuk menggunakan namespace standar C++

Penjelasan Detail:

- **namespace** adalah wadah yang mengelompokkan identifiers (nama variabel, fungsi, dll)
- **std** adalah namespace standar C++ yang berisi fungsi-fungsi seperti cout, cin, endl
- Tanpa baris ini, kita harus menulis `std::cout` bukan hanya `cout`
- Mempermudah penulisan kode

Contoh:

Tanpa using namespace std → `std::cout << "Hello";`

Dengan using namespace std → `cout << "Hello";`

FUNGSI isPrima() - Mengecek Bilangan Prima

Baris 4

DEKLARASI FUNGSI

```
bool isPrima(int n) {
```

Fungsi:

Mendefinisikan fungsi bernama isPrima yang menerima parameter dan mengembalikan nilai boolean

Penjelasan Detail:

- **bool** = tipe data return (true atau false)
- **isPrima** = nama fungsi
- **int n** = parameter input bertipe integer (bilangan yang akan dicek)
- **{** = pembuka blok kode fungsi

Cara Kerja:

Ketika dipanggil: isPrima(7) → fungsi akan mengecek apakah 7 prima → return true

Baris 5

```
if (n <= 1) return false;
```

Fungsi:

Mengecek apakah bilangan kurang dari atau sama dengan 1, jika ya maka bukan prima

Penjelasan Detail:

- **if** = kondisi percabangan
- **n <= 1** = jika n kurang dari atau sama dengan 1
- **return false** = kembalikan nilai false (bukan prima)
- Bilangan prima dimulai dari 2

Contoh:

```
isPrima(0) → return false  
isPrima(1) → return false  
isPrima(-5) → return false
```

Baris 6

```
if (n == 2) return true;
```

Fungsi:

Mengecek apakah bilangan adalah 2, jika ya maka prima

Penjelasan Detail:

- **n == 2** = jika n sama dengan 2
- **return true** = kembalikan nilai true (adalah prima)
- 2 adalah satu-satunya bilangan prima genap
- Pengecekan khusus untuk efisiensi

Fakta:

2 adalah bilangan prima terkecil dan satu-satunya bilangan prima genap

Baris 7

```
if (n % 2 == 0) return false;
```

Fungsi:

Mengecek apakah bilangan adalah genap (selain 2), jika ya maka bukan prima

Penjelasan Detail:

- **n % 2** = operasi modulo (sisa bagi n dengan 2)
- **== 0** = jika sisa bagi sama dengan 0 (habis dibagi 2 = genap)
- **return false** = bukan prima
- Semua bilangan genap selain 2 pasti bukan prima karena bisa dibagi 2

Contoh:

isPrima(4) → $4 \% 2 = 0$ → return false

isPrima(10) → $10 \% 2 = 0$ → return false

isPrima(8) → $8 \% 2 = 0$ → return false

Baris 9

```
for (int i = 3; i * i <= n; i += 2) {
```

Fungsi:

Loop untuk mengecek pembagi ganjil dari 3 sampai akar kuadrat n

Penjelasan Detail:

- **for** = perulangan

- **int i = 3** = variabel i dimulai dari 3
- **i * i <= n** = loop berjalan selama $i^2 \leq n$ (sama dengan $i \leq \sqrt{n}$)
- **i += 2** = i bertambah 2 setiap iterasi (3, 5, 7, 9, 11, ...)
- Hanya cek bilangan ganjil karena bilangan genap sudah dicek sebelumnya

Contoh untuk n = 49:

$i = 3 \rightarrow 3^2 = 9 \leq 49 \checkmark$

$i = 5 \rightarrow 5^2 = 25 \leq 49 \checkmark$

$i = 7 \rightarrow 7^2 = 49 \leq 49 \checkmark \rightarrow 49 \% 7 = 0 \rightarrow$ bukan prima!

Loop berhenti karena ketemu pembagi

Mengapa sampai $\sqrt{n}$ saja?

Karena jika $n = a \times b$ dan $a > \sqrt{n}$, maka b pasti $< \sqrt{n}$. Jadi cukup cek sampai $\sqrt{n}$ saja untuk menemukan semua pembagi.

Baris 10

```
if (n % i == 0) return false;
```

Fungsi:

Mengecek apakah n habis dibagi i, jika ya maka bukan prima

Penjelasan Detail:

- **n % i** = sisa pembagian n dengan i
- **== 0** = jika habis dibagi (sisa = 0)
- **return false** = langsung keluar dari fungsi dengan nilai false
- Jika ketemu satu pembagi saja, maka pasti bukan prima

Contoh untuk n = 15:

$i = 3 \rightarrow 15 \% 3 = 0 \rightarrow$ ketemu pembagi! $\rightarrow$ return false

Tidak perlu lanjut cek $i = 5, 7$, dst karena sudah terbukti bukan prima

Baris 12

```
return true;
```

Fungsi:

Jika lolos semua pengecekan, maka bilangan adalah prima

Penjelasan Detail:

- Baris ini hanya dieksekusi jika loop selesai tanpa menemukan pembagi
- Artinya n tidak bisa dibagi oleh bilangan apapun selain 1 dan dirinya sendiri
- **return true** = bilangan adalah prima

Contoh untuk n = 7:

Cek: $7 \% 3 = 1$ (bukan 0) ✓

Cek: $7 \% 5 = 2$ (bukan 0) ✓

Loop selesai tanpa pembagi → return true → 7 adalah prima!

Baris 13

```
}
```

Fungsi:

Menutup blok kode fungsi isPrima()

FUNGSI MAIN() - Program Utama

Baris 15

FUNGSI UTAMA

```
int main() {
```

Fungsi:

Mendefinisikan fungsi utama program (entry point)

Penjelasan Detail:

- **int** = fungsi main mengembalikan nilai integer
- **main()** = nama fungsi utama yang dijalankan pertama kali
- Setiap program C++ harus memiliki fungsi main()
- Program dimulai dari sini

Baris 16

```
int batas = 50;
```

Fungsi:

Mendeklarasikan variabel batas dengan nilai 50

Penjelasan Detail:

- **int** = tipe data integer (bilangan bulat)
- **batas** = nama variabel
- **= 50** = inisialisasi dengan nilai 50
- Variabel ini menentukan batas atas pencarian (1 sampai 50)

Bisa Dimodifikasi:

Ubah menjadi 100 untuk mencari prima 1-100

Baris 17

```
int jumlah = 0;
```

Fungsi:

Mendeklarasikan variabel counter untuk menghitung jumlah bilangan prima

Penjelasan Detail:

- **int jumlah** = variabel integer bernama jumlah
- **= 0** = diinisialisasi dengan 0
- Variabel ini akan bertambah setiap kali menemukan bilangan prima
- Di akhir program, jumlah = total bilangan prima yang ditemukan

Baris 19-21

```
cout << "=====\n";  
cout << " PROGRAM BILANGAN PRIMA (1 - 50) \n";  
cout << "=====\n\n";
```

Fungsi:

Menampilkan header/judul program ke layar

Penjelasan Detail:

- **cout** = console output (menampilkan ke layar)
- **<<** = operator insertion (memasukkan data ke cout)
- **"..."** = string literal (teks yang ditampilkan)
- **\n** = karakter newline (pindah baris baru)
- Membuat tampilan program lebih rapi dan profesional

Baris 23-24

```
cout << "Bilangan Prima antara 1 sampai " << batas << ":\n";  
cout << "-----\n";
```

Fungsi:

Menampilkan teks informasi sebelum daftar bilangan prima

Penjelasan Detail:

- Menggabungkan teks dengan nilai variabel batas
- **<< batas <<** = menampilkan nilai variabel (50)
- Garis pemisah untuk estetika

Baris 27

```
for (int i = 1; i <= batas; i++) {
```

Fungsi:

Loop untuk mengecek setiap bilangan dari 1 sampai batas (50)

Penjelasan Detail:

- **for** = perulangan
- **int i = 1** = variabel i dimulai dari 1
- **i <= batas** = loop berjalan selama $i \leq 50$
- **i++** = i bertambah 1 setiap iterasi (1, 2, 3, 4, ..., 50)
- Loop ini akan berjalan 50 kali

Alur Loop:

Iterasi 1: $i = 1$

Iterasi 2: $i = 2$

Iterasi 3: $i = 3$

...

Iterasi 50: $i = 50$

Loop selesai

Baris 28

```
if (isPrima(i)) {
```

Fungsi:

Memanggil fungsi `isPrima()` untuk mengecek apakah `i` adalah bilangan prima

Penjelasan Detail:

- **isPrima(i)** = memanggil fungsi dengan parameter `i`
- Fungsi akan return `true` jika `i` prima, `false` jika bukan
- **if (isPrima(i))** = jika hasilnya `true`, jalankan blok dalam `{ }`

- Ini adalah pemanggilan fungsi yang telah kita buat sebelumnya

Contoh:

Ketika $i = 7 \rightarrow \text{isPrima}(7) \rightarrow \text{return true} \rightarrow$ masuk ke blok if

Ketika $i = 8 \rightarrow \text{isPrima}(8) \rightarrow \text{return false} \rightarrow$ skip blok if

Baris 29

```
cout << i << " ";
```

Fungsi:

Menampilkan bilangan prima yang ditemukan ke layar

Penjelasan Detail:

- `cout << i` = menampilkan nilai `i`
- `<< " "` = menambahkan spasi setelah angka
- Angka-angka akan ditampilkan dalam satu baris dipisah spasi
- Hanya dijalankan jika `isPrima(i) = true`

Output:

2 3 5 7 11 13 17 19 23 29 31 37 41 43 47

Baris 30

```
jumlah++;
```

Fungsi:

Menambah counter jumlah setiap kali menemukan bilangan prima

Penjelasan Detail:

- `jumlah++` = increment (sama dengan `jumlah = jumlah + 1`)
- Setiap kali ketemu bilangan prima, jumlah bertambah 1
- Di akhir loop, jumlah akan berisi total bilangan prima

Proses:

Ketemu prima 2 $\rightarrow$ jumlah = 1

Ketemu prima 3 $\rightarrow$ jumlah = 2

Ketemu prima 5 $\rightarrow$ jumlah = 3

...

Selesai $\rightarrow$ jumlah = 15

Baris 31-32

```
}  
}
```



OUTPUT PROGRAM



Screenshot Output Program Setelah Di-Run

```
Terminal - Output Program  
  
=====
```

PROGRAM BILANGAN PRIMA (1 - 50)

```
=====
```

Bilangan Prima antara 1 sampai 50:

```
-----
```

2 3 5 7 11 13 17 19 23 29 31 37 41 43 47

```
-----
```

Total Jumlah Bilangan Prima: 15

```
=====
```

Process finished with exit code 0